

Introduction & Literature Review

Each year, approximately 250-260 of the 329 athletes invited to the National Football League (NFL) Combine are drafted into the league. To receive an invite to the Combine, each of these players must have an exceptional track record in their respective collegiate career that allows them to meet the criteria of the NFL's Player Selection Committee. The purpose of the NFL Combine is to collect additional data on these top prospects through one-on-one team interviews, cognitive evaluations, and physical tests, providing teams with the information needed to assess organizational fit.

This project focuses on the publicly available quantitative data collected at the Combine. It then integrates this data with relevant metrics from each player's final season of collegiate football to approximate the data driven decisions NFL teams make every year. More specifically, the goal of this work is to determine how well a player's performance metrics and college production can predict NFL draft outcomes. For simplicity, the analysis is restricted to three offensive positions: quarterback, wide receiver, and running back. For each player, the dataset includes information from the NFL Combine, including height, weight, 40-yard dash time, vertical jump height, broad jump length, three cone drill time, and bench press reps. These metrics are then combined with other relevant position based performance statistics from their final college season, sourced from the "College Football Data" open source API. This project evaluates the performance of tuned multilayer perceptrons and Bayesian logistic regression models in predicting whether a player is drafted, and identifies the most important variables driving these predictions.

This work builds on the research of two influential papers in sports analytics. The first, "Catching a Draft: On the Process of Selecting Quarterbacks in the NFL Amateur Draft" by David J. Berri and Rob Simmons, examines quarterback selection by analyzing how college performance metrics translate into NFL success. This study systematically compares draft statistics, such as passing yards, completion percentage, and other measurable traits, with actual NFL performance outcomes. Using an Ordinary Least Squares (OLS) regression model, the authors identify which features are most predictive of future success. This model provides some insight into factors that teams consider when evaluating quarterback prospects, but it lacks depth in its simplistic modeling approach.

The second paper, "The Loser's Curse: Decision Making and Market Efficiency in the National Football League Draft" by Cade Massey and Richard H. Thaler, investigates NFL decision making and shows that teams often overvalue early draft picks. By analyzing historical draft trades and generating value curves, the study compares how teams assign value to draft selections with the eventual performance of the players chosen. This paper highlights systematic biases and inefficiencies in draft decisions. Most importantly, it points out that these biases are especially prevalent when teams compete for high picks. The publication is particularly interesting because it applies an economic framework to explain why these inefficiencies persist.

Drawing on these studies, this project applies more advanced machine learning methods to model NFL draft outcomes using combine metrics and collegiate performance data. Instead of (OLS) models and economic cost curves, we employ multilayer perceptrons and Bayesian logistic regression models to predict draft selection at the positional level. This analysis then evaluates the advantages and disadvantages of each modeling approach. Similar to the publications, our work also examines feature importance to better understand the most influential variables affecting draft outcomes.

Data Collection & Machine Learning Methodology

Our data was procured from two primary sources and joined based on player name and position. The first is an open source NFL Combine performance dataset containing 4,741 entries from 2010 through 2023. The second dataset is sourced from the “College Football Data” (CFB) API, which provides comprehensive yearly statistics for all college football players.

To merge these datasets, our data cleaning process first pivots the CFB data by player name and retains only the most recent season for each player. This ensures that our dataset includes only the most relevant metrics from a player’s final year of eligibility. For both datasets, we normalize player names by removing prefixes and suffixes, eliminating special characters, and converting all text to lowercase to ensure consistency. We also standardize position labels across both datasets by mapping all variations of position names to a unified format. This allows for more reliable matching between datasets. Our matching process then applies fuzzy string matching with an 88.0% similarity threshold on both the cleaned player name and standardized position to identify corresponding records. Even with our normalization and matching procedures, our approach achieves a 52.9% match rate on our Combine data when performing an inner join between the Combine dataset and the CFB dataset. This relatively low match rate shows a limitation of our approach: the absence of a shared primary key between our two data sources. As a result, perfect alignment between datasets was not feasible without manual intervention, and our usable sample size was reduced. We then split the data into our three position sorted dataframes and filtered out unimportant columns in each by removing columns with less than 50% non null values. The reason for this is that differing positions often do not share common metrics; for example, a quarterback is unlikely to have a pass reception statistic, and a wide receiver is unlikely to have any passing yards. We then converted all the null values to zeros, heights to inches, and boolean values to their binary representations in order to create dataframes that could easily be converted into vectors and arrays. It may be important to note that in a future iteration of the project, collaborative filtering could be implemented to fill in missing values and generate a more comprehensive data set.

Moving to the modeling portion of the project, our central question is whether we can accurately predict, for each of the three specified positions, if a player will be drafted based on combine results and previous year college football statistics. To address this, we implemented two models: first, a simple multilayer perceptron with a sigmoid output activation function; and second, a Bayesian logistic regression model, where we used Metropolis–Hastings sampling to estimate the posterior distribution of the model parameters.

Starting with the multilayer perceptron model, we began by scaling each of our position data files using min-max scaling, then split our data into 80:20 train test sets, skipping a separate validation set due to the small number of samples available for training each model. After experimenting with various numbers of layers, activation functions, and epoch counts, we found that a two layer model, with both layers using ReLU activation, trained for each position produced the best overall accuracy metrics. Each position model was then trained for a varying number of epochs depending on when convergence was observed in the loss and accuracy metrics. For our quarterback prediction model, smaller layer dimensions of 64 and 32 neurons yielded superior predictions. For the other two positions, performance improved with larger layer sizes of 256 and 128 neurons for the first and second hidden layers, respectively. All models initially struggled with class imbalance, often skewing predictions toward players not being drafted. To address this, we incorporated class weight penalties, ensuring a more balanced prediction across the dataset. Each network was trained using binary cross-entropy loss.

Our second model is a Bayesian logistic regression model that answers the same question: Can we accurately predict draft outcomes for our three positions? For each position, we again used an 80:20 train test split and standardized the predictor variables using the mean and standard deviation of the training data. We modeled the draft status as a Bernoulli outcome, linking this to the predictors using a logistic function to make sure values are between 0 and 1. We placed a normal prior on the regression coefficients to discourage any extreme parameter estimates. Since we are required to implement one of our models manually, we fit this model using our own random walk Metropolis-Hastings sampler instead of a prebuilt modelling package. We ran the sampler for 25K iterations, then discarded the first 20% of samples with a 20% burn-in period and thinned 1 in 5 of the remaining draws to reduce autocorrelation. For each position, we then tuned the proposal standard deviation to make sure we get a reasonable acceptance rate. We also adjusted the classification threshold since some of the data for certain positions is more unbalanced, such as for the wide receiver position. Additionally, we tuned the prior variance to control regularization, with smaller prior variances yielding stronger shrinkage for smaller position datasets. We also chose the most relevant 7-8 metrics for each position due to sampling complexity. The final predictions we made using the posterior mean estimated draft probability for each player, which let us evaluate the model using a variety of metrics, including accuracy, precision, recall, and the combination of those with F1 scores.

Machine Learning Results & Interpretation

After building and tuning our multilayer perceptron models to predict whether a given player was drafted into the NFL, there are some interesting findings and performance variance when comparing the draft prediction accuracy between our three positions.

Starting with the quarterback model, our training dataset achieved 63.3% accuracy after 15 epochs, and our test dataset predicted 60.5% of draft outcomes correctly with an F1 score of 0.721.

Confusion Matrix

	Not Drafted	Drafted
Predicted Not Drafted	4	5
Predicted Drafted	12	22

Predicted Draft Probability by Player

Scatter plot showing Predicted Draft Probability (Y-axis, 0.48 to 0.62) versus Player Index (X-axis, 0 to 40). A dashed horizontal line at 0.50 indicates the Decision Boundary. Data points are colored green for Drafted and red for Not Drafted. Labeled players include Connor Handley, Jack Newman, Nick Fitzgerald, Sam Russell, David Coon, Zax Thomas, Blake Roberts, and Chase Linton.

Shifting to the running back model, our training dataset improved to a 69.3% accuracy after 20 epochs, and our test dataset predicted 61.8% of draft outcomes accurately with an F1 score of 0.705.

Confusion Matrix

	Not Drafted	Drafted
Predicted Not Drafted	11	16
Predicted Drafted	10	31

Predicted Draft Probability by Player

Decision Boundary: 0.50

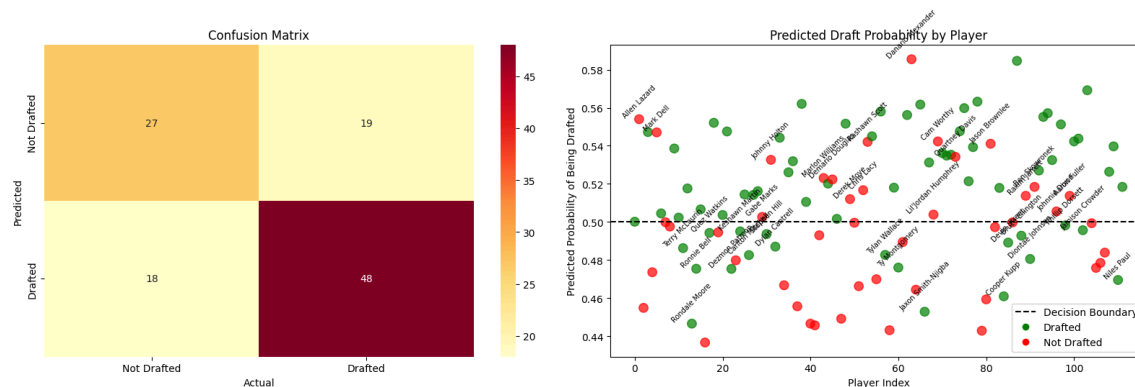
Legend:
 - Decision Boundary: Dashed line at 0.50
 - Drafted: Green dots
 - Not Drafted: Red dots

Players (approximate Player Index):
 - Not Drafted (Red): Shambaugh McNeal (~5), Mike White (~8), Kevin White (~10), Josh Johnson (~15), Jamarion McQuade (~10), Jamarion McQuade (~15), Jamarion McQuade (~20), Jamarion McQuade (~25), Jamarion McQuade (~30), Jamarion McQuade (~35), Jamarion McQuade (~40), Jamarion McQuade (~45), Jamarion McQuade (~50), Jamarion McQuade (~55), Jamarion McQuade (~60), Jamarion McQuade (~65), Jamarion McQuade (~70).
 - Drafted (Green): George Allen (~45), Daniel Scott (~50), George Allen (~55), Daniel Scott (~60), George Allen (~65), Daniel Scott (~70).

4

Looking at the wide receiver model, our training dataset achieved 67.9% accuracy after 14 epochs, and our test dataset correctly predicted 67.0% of draft outcomes with an F1 score of 0.722.

Wide Receiver Draft MLP Performance Plots

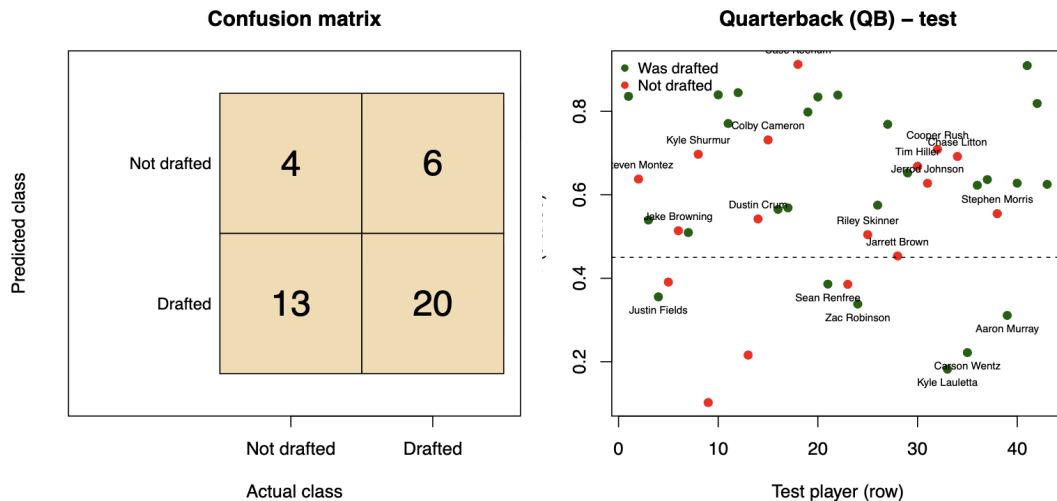


Examining our wide receiver confusion matrix, there is a slight skew toward false negatives in draft predictions, but in general, the predictions are well balanced. Looking at specific outliers, the model predicted that Danario Alexander was highly likely to be drafted. Further information shows he was projected as a mid round pick but suffered a significant knee injury ahead of the draft. This highlights that additional health related factors could be relevant in a more advanced iteration of the model. In summation, the multilayer perceptron performs reasonably well in predicting wide receiver outcomes, but could be improved with additional information.

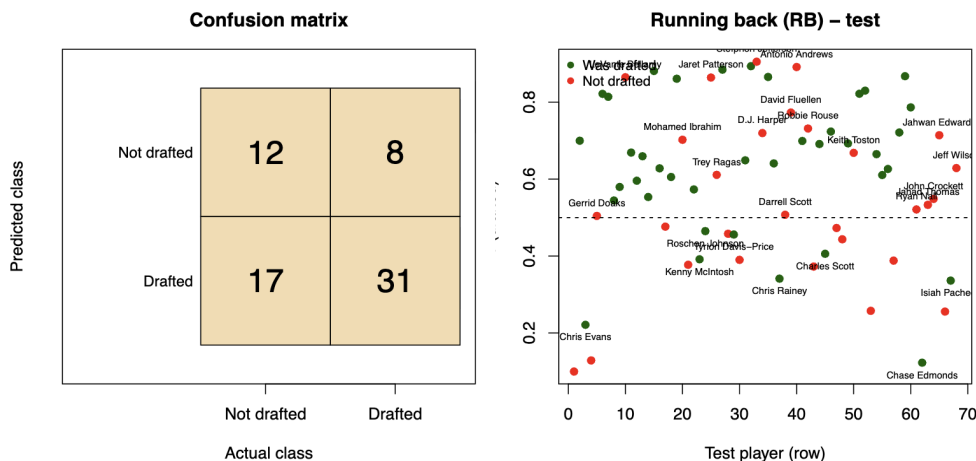
Moving to the draft prediction results from our Bayesian model. The Bayesian logistic regression model produced similar results with mixed performances across the different positions. Some positions had fairly balanced predictions, and others displayed a stronger tendency to predict that players would be drafted. Across the positions, the test accuracy ranged from the lowest of 55.8% on the Quarterback position to 65.2% in predicting the wide receiver. The test F1 score was also the highest at 0.766 for the wide receiver position and the lowest at 0.678 for Quarterbacks. Quarterback is likely the hardest position to predict, as teams are very selective in selecting them in the draft.

Starting with our quarterback model, the training accuracy was 74%, but for the test accuracy, it dropped to 55.8%, making the quarterback the weakest performing Bayesian position model for the test set. For quarterbacks, we used a proposal standard deviation of 0.14, a prior variance of 0.5, and a classification threshold of 0.45, which gave a stable acceptance rate of 37% and helped us somewhat stabilize precision. The largest posterior mean coefficients were passing yards, weight, and height at 0.981, 0.699, and 0.491, respectively. The 40-yard dash again had a large negative coefficient at -0.513. The results suggest that the most important metrics were passing yards and the physical characteristics of the quarterback. These make sense as physical size is incredibly important for the quarterback to be able to look over his offensive line to make quick throws in the NFL. Looking at the confusion matrix, the model correctly identified 20 drafted quarterbacks but only 4 undrafted quarterbacks, with 13 false positives. This shows that the model was still much better at identifying drafted quarterbacks than undrafted ones. Some players predicted to go undrafted were interesting, such as Carson Wentz and Justin Fields, who

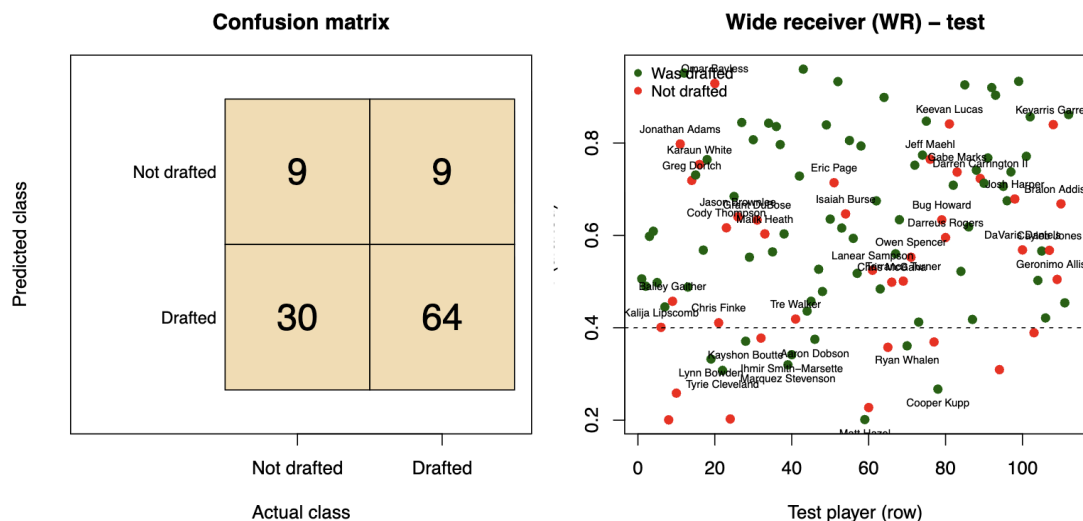
Confusion matrix **Quarterback (QB) – test**



Moving to the running back model, the training dataset achieved 71.5% accuracy, and the test dataset predicted 63.2% of the draft outcomes. Here, we used a proposal standard deviation of 0.1, prior variance of 1, and raised the threshold to 0.5 to balance precision and recall, which gave us a 35.2% acceptance rate. The largest posterior mean coefficients were rushing touchdowns, rushing yards per carry, rushing yards, and weight at 0.512, 0.369, 0.418, and 0.240, respectively. These all make sense for the most important metrics, as running backs need to be efficient while also producing touchdowns and yards. A couple of interesting players were Chase Edmonds and Isiah Pacheco, who were both drafted in later rounds but not predicted to be drafted by our model. Looking at the confusion matrix, it is slightly more balanced than some of the other position groups with 31 true positives, 12 true negatives, 17 false positives, and 8 false negatives. While the model still predicted drafted somewhat more often than undrafted, the precision and recall are more balanced than most of the other positions. This resulted in an F1 score of 0.713, which shows a solid balance. Overall, the Bayesian running back model had a decent test performance and was more balanced than other models, but not highly accurate.



Finishing with the wide receiver model, the training dataset reached an accuracy of 66.1%. The test dataset correctly predicted 65.2% of the draft outcomes. We use a proposal standard deviation of 0.08, a prior variance of 1, and a classification threshold of 0.4, which gave us a solid acceptance rate of 31.7% while still having good recall and somewhat decent precision. The most influential posterior mean coefficients were receiving yards, receiving touchdowns, and vertical jump at 0.565, 0.393, and 0.142. Again, the 40-yard dash time had a negative coefficient of -0.134. These posterior mean coefficients generally make sense, as receiving yards and receiving touchdowns are the most important metrics in what gets a wide receiver drafted. It was interesting that vertical jump is a metric that had a significant positive effect, as it is a more uncommon metric talked about when considering a player, unless the player is a significant outlier. Looking at the test confusion matrix, the model correctly guessed 64 drafted players but also had 30 undrafted players as drafted, showing a significant false positive imbalance. One interesting prediction for not getting drafted was Cooper Kupp, who has had an incredible career, winning the triple crown and Super Bowl in the 2021 season. Overall, the wide receiver Bayesian model performs reasonably well and has our highest F1 score of 0.766. It seemed to find several important draft signals, but it tends to overpredict which players will get drafted.



Conclusions & Future Work

After developing our two models for our three offensive positions and analyzing individual results, both the multilayer perceptron and Bayesian logistic regression models demonstrated their own strengths and weaknesses. This begs the question: what is a more appropriate choice for predicting position level NFL draft outcomes?

When comparing results, the multilayer perceptron models provide marginally better prediction accuracy for two of the three positions. The multilayer perceptron achieved test accuracies of 60.5% for quarterbacks, 67.2% for wide receivers, and 61.8% for running backs, compared to 55.8%, 65.2%, and 63.2% for the Bayesian logistic regression model, respectively. F1 scores were relatively similar between the two modeling types, with a slight edge going to the Bayesian Logistic Regression model for two of the three positions.

However, the multilayer perceptron's improved accuracy performance came at some cost to interpretability, and the variable weight magnitudes contributing to the model were not feasible to interpret, making analysis of feature importance for each position limited to the Bayesian models. The Bayesian logistic regression model weights made sense in the context of important factors for a given position, and the model did not suffer from the same overfitting challenges the multilayer perceptron models struggled with.

Shifting back to the multilayer perceptron, the ease of implementation, better accuracy, and increased tunability make a strong argument for it being a better model for binary classification. The multilayer perceptron also exhibited a more proportional balance of predictions when comparing confusion matrices. This is likely due to its added class weight penalties. Given this information, the multilayer perceptron is our choice for this classification task. It is important to acknowledge that for a more detailed feature analysis task or intensive modeling tasks with distribution outputs, the Bayesian logistic regression model may be a better choice due to its increased feature input modeling and probabilistic output.

Considering possible improvements and extensions for this project, a significant limiting factor was the small position dataset sizes observed after matching the two datasets. To address this, it would likely make sense to go through a more intensive manual matching process in order to train and test our models with more comprehensive datasets. There were a significant number of null values in our combined dataset, so adding either item-based or column-based collaborative filtering approaches would likely yield a more comprehensive dataset. For future work, we would likely want to procure more historical data in order to improve the training ability of both models. Our outliers also provided some insight that our predictions could benefit from additional metrics, including injury history, college strength of schedule, or college team division.

In summation, our two models performed effectively for the position-level binary classification task and gave strong insights into factors that affect a given player's draftability. A key takeaway from our analysis is that there is a clear tradeoff between predictive performance and interpretability in machine learning approaches.

Appendix Guide

- Page 10: References for sources used in the project.
- Pages 11-12: Python multi-layer perceptron code snippets for each position.
- Pages 13-14: Bayesian logistic regression and sampling code for each position.

References

- Berri, David J., and Rob Simmons. "Catching a Draft: On the Process of Selecting Quarterbacks in the National Football League Amateur Draft." *Journal of Productivity Analysis* 35, no. 1 (2011): 37–49. <https://doi.org/10.1007/s11123-009-0154-6>.
- Massey, Cade, and Richard H. Thaler. "The Loser's Curse: Decision Making and Market Efficiency in the National Football League Draft." *Management Science* 59, no. 7 (2013): 1479–1495. <https://doi.org/10.1287/mnsc.1120.1657>.
- "2025 NFL Scouting Combine: League Releases Names of 329 Prospects Invited to Indianapolis." *CBS Sports*, February 5, 2025.
<https://www.cbssports.com/nfl/news/2025-nfl-scouting-combine-league-releases-names-of-329-prospects-invited-to-indianapolis/>.
- "College Football Data API." *College Football Data*. Accessed March 3, 2026.
<https://api.collegefootballdata.com>.
- Shaw, Thomas. "NFL Combine Performance Dataset." *Kaggle*. Accessed April 10, 2026.
<https://www.kaggle.com/datasets/thomasshaw/nfl-combine-performance-dataset>.
- Robinston, Michael David. "Fuzzy Matching for Million Row Address Dataset (Large Dataset) with RapidFuzz and Splink." *Medium*, August 20, 2022.
<https://medium.com/@bewin4u/fuzzy-matching-for-million-row-address-dataset-with-rapidfuzz-and-splink-b704eaf1fda9>. Accessed April 13, 2026.

Python Model Code:

Quarterback Position:

```
qb_df = pd.read_csv("data/qb_data.csv")
y_qb = qb_df["Drafted"].to_numpy()
names = qb_df["Player"].values

X_qb = qb_df.drop(columns=["Player", "Pos", "Pick",
"Drafted"]).to_numpy()
X_qb = MinMaxScaler().fit_transform(X_qb)

X_train, X_test, y_train, y_test, names_train, names_test =
train_test_split(X_qb, y_qb, names, test_size=0.2,
random_state=42)
inpx = Input(shape=(X_train.shape[1],))

hid_layer = Dense(64, activation='relu')(inpx)
hid_layer2 = Dense(32, activation='relu')(hid_layer)
out_layer = Dense(1, activation='sigmoid')(hid_layer2)
model = Model([inpx], out_layer)

model.compile(optimizer=SGD(), loss=binary_crossentropy,
metrics=['accuracy'])
class_weights = compute_class_weight('balanced',
classes=np.array([0, 1]), y=y_train.ravel())
class_weight_dict = {0: class_weights[0], 1: class_weights[1]}

model.fit(X_train, y_train, epochs=15,
class_weight=class_weight_dict)
```

Running Back Position:

```
rb_df = pd.read_csv("data/rb_data.csv")
y_rb = rb_df["Drafted"].to_numpy()
names = rb_df["Player"].values

X_rb = rb_df.drop(columns=["Player", "Pos", "Pick",
"Drafted"]).to_numpy()
X_rb = MinMaxScaler().fit_transform(X_rb)

X_train, X_test, y_train, y_test, names_train, names_test =
train_test_split(X_rb, y_rb, names, test_size=0.2,
random_state=42)
inpx = Input(shape=(X_train.shape[1],))
```

```

hid_layer = Dense(256, activation='relu')(inpx)
hid_layer2 = Dense(128, activation='relu')(hid_layer)
out_layer = Dense(1, activation='sigmoid')(hid_layer2)
model = Model([inpx], out_layer)

model.compile(optimizer=SGD(), loss=binary_crossentropy,
metrics=['accuracy'])
class_weights = compute_class_weight('balanced',
classes=np.array([0, 1]), y=y_train.ravel())
class_weight_dict = {0: class_weights[0], 1: class_weights[1]}

model.fit(X_train, y_train, epochs=25,
class_weight=class_weight_dict)

```

Wide Receiver Position:

```

wr_df = pd.read_csv("data/wr_data.csv")
y_wr = wr_df["Drafted"].to_numpy()
names = wr_df["Player"].values

X_wr = wr_df.drop(columns=["Player", "Pos", "Pick",
"Drafted"]).to_numpy()
X_wr = MinMaxScaler().fit_transform(X_wr)

X_train, X_test, y_train, y_test, names_train, names_test =
train_test_split(X_wr, y_wr, names, test_size=0.2,
random_state=42)
inpx = Input(shape=(X_train.shape[1],))

hid_layer = Dense(256, activation='relu')(inpx)
hid_layer2 = Dense(128, activation='relu')(hid_layer)
out_layer = Dense(1, activation='sigmoid')(hid_layer2)
model = Model([inpx], out_layer)

model.compile(optimizer=SGD(), loss=binary_crossentropy,
metrics=['accuracy'])
class_weights = compute_class_weight('balanced',
classes=np.array([0, 1]), y=y_train.ravel())
class_weight_dict = {0: class_weights[0], 1: class_weights[1]}

model.fit(X_train, y_train, epochs=20,
class_weight=class_weight_dict)

```

R Model Code:

```
sigmoid <- function(linear_predictor) {
  ifelse(
    linear_predictor >= 0,
    1 / (1 + exp(-linear_predictor)),
    exp(linear_predictor) / (1 + exp(linear_predictor))
  )
}

log_likelihood_logit <- function(coefficients, design_matrix,
drafted_binary) {
  linear_predictor <- as.vector(design_matrix %*% coefficients)
  sum(drafted_binary * linear_predictor -
loglp(exp(linear_predictor)))
}

log_prior <- function(coefficients, prior_variance_scale) {
  -0.5 * sum(coefficients^2) / prior_variance_scale
}

log_posterior <- function(coefficients, design_matrix,
drafted_binary, prior_variance_scale) {
  log_likelihood_logit(coefficients, design_matrix,
drafted_binary) +
  log_prior(coefficients, prior_variance_scale)
}

run_mh <- function(design_matrix, drafted_binary, n_iter,
proposal_sd, prior_variance_scale, seed = 42) {
  set.seed(seed)
  n_coefficients <- ncol(design_matrix)
  current_coefficients <- rep(0, n_coefficients)
  mcmc_sample_matrix <- matrix(NA_real_, nrow = n_iter, ncol =
n_coefficients)
  colnames(mcmc_sample_matrix) <- colnames(design_matrix)
  log_posterior_current <- log_posterior(
    current_coefficients, design_matrix, drafted_binary,
prior_variance_scale
  )
  n_accepted <- 0L

  for (iteration in seq_len(n_iter)) {
    proposed_coefficients <- current_coefficients +
rnorm(n_coefficients, mean = 0, sd = proposal_sd)
    log_posterior_proposed <- log_posterior(
```

```

        proposed_coefficients, design_matrix, drafted_binary,
prior_variance_scale
    )
    if (log(stats::runif(1)) < log_posterior_proposed -
log_posterior_current) {
        current_coefficients <- proposed_coefficients
        log_posterior_current <- log_posterior_proposed
        n_accepted <- n_accepted + 1L
    }
    mcmc_sample_matrix[iteration, ] <- current_coefficients
}

acceptance_rate <- as.numeric(n_accepted) / n_iter
cat(sprintf("Acceptance rate: %.3f\n", acceptance_rate))
list(chain = mcmc_sample_matrix, acceptance_rate =
acceptance_rate)
}

thin_chain <- function(full_chain, burn_in, thin) {
    full_chain[seq(burn_in + 1, nrow(full_chain), by = thin), ,
drop = FALSE]
}
default_tau_sq <- 1
n_iter <- 25000
burn_in <- 5000
thin <- 5

position_config <- list(
    WR = list(
        proposal_sd = 0.08,
        tau_sq = default_tau_sq,
        preds = c(
            "Height", "Weight", "40yd", "Vertical",
            "receiving_YDS", "receiving_TD", "receiving_YPR",
"receiving_REC"
        )
    )
)

```